An Industrial Internship Report

submitted by;

Rohan Yenigalla

(23BEC0027)

in partial fulfillment for the award of the degree of

B.TECH

in

ELECTRONICS AND COMMUNICATIONS ENGINEERING

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Table of Contents

Abstract *4*

Acknowledgement *5*

Introduction *6*

About HAL: **6**

About MCSRDC: **7**

Problem Statement: **8**

Research Objectives: **8**

Technical Description *9*

Components of an Aircraft: **9**

Directional Gyro (DG): 9
Vertical Speed Indicator (VSI): 9
Turn Coordinator: 10
Altimeter: 10
Attitude Indicator (AI): 11
VOR Receiver with Glideslope: 12
GPS (Global Positioning System): 13
INGPS (Integrated Navigation GPS): 13
SMFDs (Smart Multi-Function Displays): 13
DMG (Digital Map Generator): 14
EFIS (Electronic Flight Instrument System): 15
HUDs (Head-Up Displays): 15
HMDs (Helmet-Mounted Displays): 16
CMDS (Countermeasure Dispensing System): 16
IFF (Identification Friend or Foe): 17

Mission Computer: **18**

Communication Protocols: **20**

UART Protocol: 20
SPI Protocol: 21
I2C Protocol: 22

Rigs and Simulators **24**

Rigs: 24
Simulators: 26

PCB Designing: **30**

Smart Fusion Kit: **35**

Software's Used: **37**

Libero SoC Design Suite: 37
Soft Console: 38

LED blinking using Verilog and *41*

Results *41*

Clock Generation: **41**

Need of Clock: 41
Usage of the Clock for the LED Verilog Code: 42

Verilog Code for 1 sec Blinking of LED: **42**

Verilog Code for 1,2 and 4 sec Blinking of LED: **44**

Coding MSS using C *46*
1. Why is it used? 46
Choice based blinking of LED with Pattern *50*
(1,2 and 4 Sec Interval) *50*
APB Bridge/Interconnect: 50
Verilog Code: **51**
Soft Console: **53**
MSS configuration: **54**
Challenges and Solutions *57*
Technical Challenges: **57**
Solutions: **57**
Impact and Contributions *58*
Personal Impact: **58**
Contributions: **58**
Conclusion and Future Work *59*
Conclusion: **59**
Future Work: **60**
References: *61*
CERTIFICATE *62*

Abstract

The internship experience at Hindustan Aeronautics Limited (HAL) in the Mission & Combat System Research & Design Centre (MCSRDC) Division provided an in-depth exposure to cutting-edge embedded systems and electronic technologies relevant to aircraft components and systems. During the internship, I engaged with various aspects of hardware and software development, including System on Chip (SoC) technologies, communication protocols, and the use of rigs and simulators. This internship offered a unique opportunity to apply theoretical knowledge to real-world challenges and gain hands-on experience with cutting-edge embedded systems.

I benefited from the guidance and expertise of experienced engineers at HAL. Through mentorship and collaboration, I developed a deeper understanding of FPGA and Microcontroller Subsystem (MSS) integration using the APB bus. I gained proficiency in utilizing tools and software such as Libero and Soft Console for Verilog and C programming, respectively. These tools enabled me to efficiently design, simulate, and verify circuits, including LED blinking patterns with various intervals and temperature-based voltage regulation.

Additionally, the internship enhanced my problem-solving skills, creativity, and adaptability in applying embedded systems solutions to complex aerospace challenges. My internship at HAL's MCSRDC Division was a rewarding and enriching experience. Through hands-on projects, mentorship, and collaboration, I gained valuable skills and knowledge that will serve me well in my future endeavours in the field of aerospace electronics.

Acknowledgement

I am sincerely grateful to Hindustan Aeronautics Limited (HAL) for providing me with the invaluable opportunity to participate in a project internship program at the MCSRDC Division. This experience, conducted from May 12th to June 27th, has been immensely enriching and has significantly contributed to my understanding of embedded systems and electronics in an aerospace context.

I extend my heartfelt gratitude to the entire team at HAL's MCSRDC Division for their guidance, support, and mentorship throughout my internship. Their expertise and willingness to share knowledge have been instrumental in my professional development.

I would also like to thank VIT university ,VELLORE for their support and for facilitating this valuable internship opportunity. The hands-on experience gained in a conductive learning environment has been invaluable.

This internship has not only enhanced my technical skills but has also provided me with insights into the critical role of embedded systems in modern aircraft design and operation. I am confident that the knowledge and experience gained during this internship will serve as a solid foundation for my future career endeavours.

Once again, I would like to thank Hindustan Aeronautics Limited (HAL) for this incredible opportunity and their commitment to nurturing the next generation of professionals.

Sincerely,

Rohan Yenigalla
(23BEC0027)

Introduction

About HAL:

Hindustan Aeronautics Limited (HAL) is an Indian public sector aerospace and defence company, headquartered in Bengaluru. Established on December 23, 1940, by Walchand Hirachand, it is one of the oldest and largest aerospace and defence manufacturers globally.

HAL is primarily engaged in the design, development, manufacture, repair, overhaul, upgrade, and servicing of a wide range of aerospace products. These include:

- **Aircraft:** Such as HAWK, Light Combat Aircraft (LCA) Tejas, SU-30 MKI, Intermediate Jet Trainer (IJT), DORNIER, and HTT-40.
- **Helicopters:** Including Dhruv, Cheetah, Chetak, Lancer, Cheetal, Rudra, Light Combat Helicopter (LCH), and Light Utility Helicopter (LUH).
- **Aero-engines, avionics, accessories, and aerospace structures**.

The company also has 11 dedicated Research and Development (R&D) centres and 21 manufacturing divisions spread across India. HAL plays a crucial role in modernizing the Indian Air Force and is involved in various indigenous aircraft projects, strategic partnerships, and the development of cutting-edge technologies.

About MCSRDC:

The Mission & Combat Systems Research & Development Centre (MCSRDC) is a key R&D division of Hindustan Aeronautics Limited (HAL), headquartered in Bengaluru, India. This centre is specifically involved in the design and development of mission and combat systems for various aerial vehicles, including fixed-wing and rotary-wing aircraft.

MCSRDC focuses on advanced aerospace technologies and often undertakes projects involving cutting-edge embedded systems, communication protocols, and simulation technologies. For example, past work has included the procurement and integration of advanced flight simulator components like G-seats and G-suit control systems.

Essentially, MCSRDC drives innovation in critical areas of aerospace technology, focusing on the sophisticated electronic and embedded systems that are integral to the functionality, safety, and combat effectiveness of modern aircraft. The division's work contributes significantly to HAL's overarching goal of enhancing India's self-reliance in aerospace and defence.

As a research and development arm of HAL, MCSRDC plays a vital role in enhancing India's indigenous defence capabilities by contributing to the development of sophisticated aircraft and associated systems.

Problem Statement:

The objective of the LED blinking project was to design, implement, and verify a system for controlling LED illumination with varying intervals using Verilog on the Smart Fusion Kit. This project aimed to provide hands-on experience with FPGA programming and embedded system development, specifically focusing on hardware description languages, simulation techniques, and the practical application of these concepts for basic output control.

Research Objectives:

- **To gain practical proficiency in Hardware Description Languages (HDLs):** Specifically, to become proficient in writing, simulating, and synthesizing Verilog code for basic digital logic functions, such as controlling LED states.
- **To understand FPGA architecture and development flow:** To familiarize oneself with the process of configuring Field-Programmable Gate Arrays (FPGAs) using tools like Libero, including pin mapping and synthesis.
- **To implement and verify timing-sensitive designs:** To develop a system that accurately controls the blinking intervals of LEDs, demonstrating an understanding of clocking and timing constraints in digital circuits.
- **To integrate FPGA and Microcontroller Subsystem (MSS) functionalities:** To learn how to combine hardware logic (FPGA) with software control (MSS using C programming), particularly through interfaces like the APB bus on the Smart Fusion Kit.
- **To develop debugging and verification skills in embedded systems:** To effectively use tools like Tera Term for observing system outputs and verifying the correct operation of both hardware and software components.

Technical Description

Components of an Aircraft:

Directional Gyro (DG):

- **Function:** Displays the aircraft's heading (direction relative to North). Unlike a magnetic compass, it is much more stable and not subject to magnetic deviations, making it easier to read during turns and in turbulent air.
- **How it works:** Uses a gyroscopic principle to maintain a fixed orientation in space, which is then referenced against a compass card to show heading. It needs to be periodically reset and aligned with the magnetic compass due to "precession" (drift).

Vertical Speed Indicator (VSI):

- **Function:** Shows the aircraft's instantaneous rate of climb or descent in feet per minute (fpm).
- **How it works:** Operates on the principle of differential pressure. It has a diaphragm connected to the static port (ambient air pressure) and a calibrated leak to the aircraft's static system. When the aircraft changes altitude, the pressure inside the diaphragm changes faster than the pressure in the case, causing the diaphragm to expand or contract, moving the needle.

Turn Coordinator:

- **Function:** Provides information on the aircraft's rate of turn and quality of turn (whether the turn is coordinated or "slipping/skidding").
- **How it works:**
    1. **Turn Rate:** A small miniature aircraft symbol banks in proportion to the rate of turn. This is driven by an electrically powered gyroscope.
    2. **Slip/Skid Indicator (the "ball"):** A curved tube containing a ball. If the turn is coordinated, the ball stays cantered. If the aircraft is slipping (insufficient rudder for the bank), the ball moves to the inside of the turn. If skidding (too much rudder for the bank), the ball moves to the outside. "Step on the ball" is a common mnemonic for correcting an uncoordinated turn.

Altimeter:

- **Function:** Measures and displays the aircraft's altitude, typically in feet above a reference level.
- **Types:** There are two different types:
    1. **Radio Altimeter (RA) / Radar Altimeter:**
        - **Function:** Measures the **absolute height** of the aircraft above the terrain directly below it (Above Ground Level - AGL). This is in contrast to a barometric altimeter, which measures altitude above a set pressure level.

- **How it works:** Transmits a radio signal downwards to the ground and measures the time it takes for the signal to reflect back to the aircraft. Since radio waves travel at a known speed, this time delay can be converted into an accurate height measurement. Radio altimeters are crucial for low-altitude flight, landing approaches, and terrain-following systems.

2. **Baro Altimeter (Barometric Altimeter):**
   - **Function:** As previously explained, this instrument measures altitude based on static air pressure, displaying height above a pre-set pressure level (e.g., Mean Sea Level - MSL, or airfield elevation).
   - **How it works:** Contains an aneroid capsule that expands and contracts with changes in atmospheric pressure. This mechanical movement is translated into altitude readings. Pilots set the local barometric pressure (altimeter setting) into the Kollsman window to ensure accurate readings relative to a specific ground reference.

Attitude Indicator (AI):

- **Function:** Shows the aircraft's attitude relative to the horizon, indicating pitch (nose up/down) and bank (wings level, left bank, right bank). It's one of the most fundamental instruments for instrument flight.

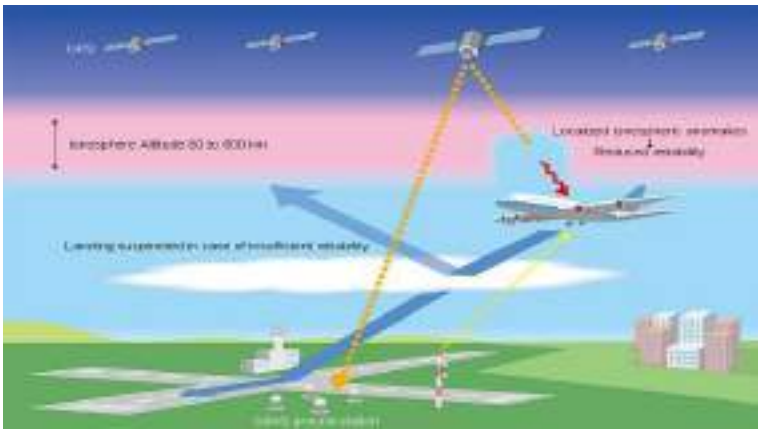
- **How it works:** Uses a gyroscope that is spun rapidly by air pressure (vacuum system) or electricity. The gyroscope remains fixed in space, and the instrument's display (a miniature aircraft against a symbolic horizon bar) moves relative to it, providing an artificial horizon.

VOR Receiver with Glideslope:

- **Function:**
    - **VOR (VHF Omnidirectional Range):** A radio navigation system that provides aircraft with an angular position from a transmitting station. The VOR receiver in the aircraft interprets these signals and displays a course deviation indicator (CDI) to show if the aircraft is on the selected course.
    - **Glideslope:** A component of an Instrument Landing System (ILS). It provides vertical guidance (angle of descent) to the runway. When a VOR receiver is equipped with glideslope capability, it can also display vertical deviation from the ideal descent path during an ILS approach.
- **How it works:** Receives radio signals from ground-based VOR stations and, for glideslope, from separate glideslope transmitters near the runway threshold. The instrument then translates these signals into visual indications for the pilot.

GPS (Global Positioning System):

- **Function:** A satellite-based navigation system that provides precise position, velocity, and time information anywhere on Earth.
- **How it works:** The GPS receiver in the aircraft receives signals from multiple GPS satellites orbiting Earth. By accurately measuring the time it takes for these signals to reach the receiver, the system can triangulate the aircraft's exact position.



INGPS (Integrated Navigation GPS):

- **Function:** This term often refers to a more advanced, integrated navigation system that combines Global Positioning System (GPS) data with other navigation sensors (like Inertial Reference Systems - IRS, or VOR/DME) for highly accurate position and navigation information. It typically manages flight plans, calculates fuel consumption, and optimizes routes.
- **How it works:** Processes data from multiple sources (GPS satellites, internal inertial sensors, radio navigation aids) through a sophisticated computer system to provide precise position, velocity, and attitude information. It can automatically tune radios and display complex navigational information.

SMFDs (Smart Multi-Function Displays):

- **Function:** This term likely refers to **Software-driven Multi-Function Displays**. They are advanced, highly customizable digital screens in the cockpit that can display a wide variety of information, replacing many traditional analog gauges. They are "smart" because their content can be dynamically reconfigured by the pilot or automatically by the flight

management system based on the phase of flight or mission requirements.

- **How it works:** They are high-resolution screens (LCDs, OLEDs) connected to the aircraft's avionics bus and mission computer. They receive data from various sensors and systems, which is then processed and rendered graphically. Pilots can typically select different display modes, such as navigation maps, weather radar, engine parameters, system schematics, or tactical information.

DMG (Digital Map Generator):

- **Function:** Generates and displays digital moving maps in the cockpit, providing pilots with a real-time representation of their aircraft's position relative to terrain, geographical features, waypoints, airfields, and airspace boundaries.
- **How it works:** Stores vast amounts of geographical data (terrain elevation, cultural features, aeronautical charts) digitally. It uses the aircraft's precise navigation data (from GPS, INS, etc.) to overlay the aircraft's current position and track onto the map, often with customizable zoom levels and overlays (e.g., weather, traffic).

EFIS (Electronic Flight Instrument System):

- **Function:** An integrated system that uses electronic displays to present flight instrument information (attitude, altitude, airspeed, heading, vertical speed) rather than traditional mechanical gauges. It's the core of a "glass cockpit."
- **How it works:** Replaces the classic "six-pack" of primary flight instruments with two main display units:
    - **Primary Flight Display (PFD):** Shows attitude, airspeed, altitude, heading, and vertical speed, often combined with navigation information.
    - **Multi-Function Display (MFD):** Can display navigation maps, engine parameters, weather, system synoptics, and other information, often in conjunction with SMFD capabilities.
- EFIS enhances situational awareness, reduces pilot workload, and improves reliability compared to older analog systems.

HUDs (Head-Up Displays):

- **Function:** Projects critical flight information (airspeed, altitude, heading, targeting cues, warning messages) onto a transparent screen directly in the pilot's forward field of view, so they don't have to look down at the instrument panel. This allows the pilot to keep their "head up and eyes out," especially vital during critical phases of flight like take off, landing, or combat.

- **How it works:** Uses a projector to display collimated (focused at infinity) images onto a combiner glass, which is transparent. The information appears to float in space ahead of the aircraft.

HMDs (Helmet-Mounted Displays):

- **Function:** Projects flight and mission critical information directly onto the pilot's helmet visor, allowing them to see data and targeting cues regardless of where they are looking. This is particularly valuable in fighter aircraft, enabling "off-boresight" targeting where the pilot can simply look at a target and lock on a weapon.
- **How it works:** Similar to HUDs, but the projection system is integrated into the helmet. It tracks the pilot's head movements to ensure the projected information remains aligned with their field of view.

CMDS (Countermeasure Dispensing System):

- **Function:** Designed to protect the aircraft from missile threats by deploying defensive countermeasures such as flares (to decoy heat-seeking missiles) and chaff (to decoy radar-guided missiles).

- **How it works:** Integrates with missile warning systems (MWS) or radar warning receivers (RWR). When a threat is detected, the CMDS can be manually activated by the pilot or configured to automatically dispense flares and/or chaff in specific sequences to break the missile's lock.

IFF (Identification Friend or Foe):

- **Function:** An electronic system used by military and civilian air traffic control to distinguish between friendly and potentially hostile aircraft. It also aids in air traffic management by allowing ground controllers to "interrogate" aircraft and receive a coded response.
- **How it works:** An interrogator (on the ground or another aircraft) sends out a coded radio signal. A transponder on the aircraft receives this signal and sends back a unique coded reply, which is then displayed on the interrogator's radar screen. This code identifies the aircraft (e.g., its callsign or mission).

Mission Computer:

A Mission Computer is a specialized, high-performance computer system found in advanced aircraft, particularly military aircraft (fighters, bombers, transport, and surveillance) and increasingly in complex civilian aircraft. It serves as the **central brain** for managing and integrating various aircraft systems related to its specific mission.

Here's a breakdown of its key functions and characteristics:

Core Functions:

- **Data Fusion and Processing:** It receives and integrates data from a multitude of sensors (radar, electro-optical/infrared, electronic warfare systems, GPS, inertial navigation systems, etc.), processes this raw data, and presents a comprehensive tactical picture to the pilot.
- **Weapon Management:** It manages the aircraft's weapon systems, including targeting, release parameters, inventory tracking, and coordinating with the pilot's controls. This involves calculating launch zones, ballistic trajectories, and release sequences for missiles, bombs, and guns.
- **Navigation and Flight Path Management:** While dedicated flight control computers handle basic aircraft stability, a mission computer often works in conjunction to provide advanced navigation capabilities, optimize flight paths for mission objectives, and manage waypoints specific to the mission.
- **Tactical Displays and Human-Machine Interface (HMI):** It generates and manages the information displayed on the multi-function displays (MFDs) in the cockpit, providing pilots with mission-critical data, maps, sensor feeds, and weapon status in an intuitive format. It also processes pilot inputs from joysticks, throttles, and control panels.
- **Electronic Warfare (EW) Management:** It can manage and integrate electronic warfare systems, including radar warning receivers, jammers, and chaff/flare dispensers, to enhance aircraft survivability.
- **Communication Management:** Oversees and manages various communication systems, ensuring secure and efficient data and voice links with other aircraft, ground stations, and command centres.

- **Resource Management:** Monitors and optimizes the use of aircraft resources relevant to the mission, such as fuel, power, and sensor modes.

Key Characteristics:

- **Real-time Processing:** Mission computers are designed for extremely fast, real-time processing to handle rapidly changing tactical situations and sensor data.
- **High Reliability and Redundancy:** Given their critical role, they are built with high levels of reliability, fault tolerance, and often include redundant systems to ensure continuous operation even in case of component failure.
- **Ruggedized Design:** They are designed to withstand extreme environmental conditions found in aircraft, including wide temperature ranges, vibrations, G-forces, and electromagnetic interference.
- **Modular and Expandable:** Often designed with a modular architecture to allow for upgrades and integration of new sensors, weapons, and software capabilities as missions evolve.
- **Specialized Operating Systems:** They typically run specialized real-time operating systems (RTOS) optimized for deterministic performance and safety.

In essence, the mission computer is what transforms a flying platform into a fully functional "mission-capable" aircraft, enabling it to execute complex tasks beyond simple flight. It's the brain that processes information, makes decisions (or assists the pilot in making them), and controls the various systems necessary to achieve the aircraft's operational objectives.

Communication Protocols:

UART Protocol:

UART is a hardware module that handles serial communication by converting parallel data into serial data for transmission, and serial data back into parallel data for reception.

- **Nature:** Asynchronous. There is no shared clock signal between the communicating devices. Instead, both the transmitter and receiver must agree on a pre-defined data transmission rate (baud rate).
- **Wiring:** Typically uses two dedicated data lines plus a common ground:
    - **TX (Transmit):** Data output from the transmitting device.
    - **RX (Receive):** Data input for the receiving device.
- **Communication Type:** Point-to-point, full-duplex communication (simultaneous sending and receiving). While multiple devices can be connected to a UART bus, they all receive the same data, requiring software-level filtering for specific recipients.
- **Data Framing:** Data is transmitted in frames, which include:
    - A **Start Bit** (low signal) to mark the beginning.
    - **Data Bits** (5 to 9 bits, commonly 8 bits).
    - An optional **Parity Bit** for basic error detection.
    - One or two **Stop Bits** (high signal) to mark the end.
- **Speed:** Relatively slower compared to SPI and I2C, typically ranging from a few kilobits per second (kbps) up to 115.2 kbps, though some implementations can reach a few megabits per second (Mbps).
- **Advantages:** Simple to implement, low hardware overhead.
- **Disadvantages:** No inherent addressing for multiple devices, relies on precise baud rate matching, no built-in error checking beyond parity.
- **Typical Applications:**
    - Debugging interfaces (e.g., serial console output).

- Communication with GPS modules, Bluetooth modules, Wi-Fi modules.
- Interfacing with modems (e.g., RS-232, RS-485).
- Simple data logging.

SPI Protocol:

SPI is a synchronous serial data protocol widely used for short-distance, high-speed communication between a microcontroller and peripheral devices.

- **Nature:** Synchronous. Communication is synchronized by a shared clock signal generated by the master device.
- **Wiring:** Uses four dedicated lines plus a common ground, enabling full-duplex communication:
  - **SCLK (Serial Clock):** Output from the master, synchronizes data transfer.
  - **MOSI (Master Out Slave In):** Data line from master to slave.
  - **MISO (Master In Slave Out):** Data line from slave to master.
  - **SS/CS (Slave Select/Chip Select):** An active-low signal from the master to enable communication with a specific slave. Each slave requires its own dedicated SS/CS line.
- **Communication Type:** Single-master, multiple-slave architecture. The master controls which slave it communicates with by asserting the corresponding SS/CS line.
- **Data Transfer:** Data is typically exchanged in 8-bit or 16-bit shifts, synchronized with the SCLK.
- **Speed:** High-speed, often reaching tens of megahertz (MHz), making it the fastest among these three protocols (e.g., 10 Mbps to over 100 Mbps in some cases).
- **Advantages:** High speed, full-duplex communication, flexible data size, no complex addressing.

- **Disadvantages:** Requires more wires than I2C (especially with many slaves), no inherent flow control or acknowledgment (ACK/NACK) mechanism.
- **Typical Applications:**
    - Interfacing with flash memory (e.g., SD cards, EEPROMs).
    - Controlling display modules (LCDs, OLEDs).
    - High-speed sensor data acquisition (e.g., accelerometers, gyroscopes).
    - Connecting to ADCs (Analog-to-Digital Converters) and DACs (Digital-to-Analog Converters).

I2C Protocol:

I2C (pronounced "eye-squared-see" or "eye-two-see") is a multi-master, multi-slave serial bus developed by Philips (now NXP Semiconductors) for connecting low-speed peripheral ICs to processors.

- **Nature:** Synchronous. Communication is synchronized by a shared clock signal (SCL).
- **Wiring:** Uses only two bidirectional open-drain lines plus a common ground, both requiring pull-up resistors:
    - **SCL (Serial Clock Line):** Synchronizes data transfer.
    - **SDA (Serial Data Line):** Bidirectional line for data transmission and reception.
- **Communication Type:** Multi-master, multi-slave architecture. Devices on the bus are addressed using unique 7-bit (or 10-bit) addresses. Master devices can arbitrate for control of the bus.
- **Data Transfer:** Data is transmitted in bytes, followed by an acknowledgment (ACK/NACK) bit from the receiving device.
- **Speed:** Moderate speed, with various modes:
    - Standard Mode: 100 kbps
    - Fast Mode: 400 kbps

    - Fast-mode Plus: 1 Mbps
    - High-Speed Mode: 3.4 Mbps
- **Advantages:** Minimal wiring (only two data lines), supports multiple masters and slaves, built-in acknowledgment (ACK/NACK) for reliable communication, good for short-distance communication on a PCB.
- **Disadvantages:** Slower than SPI, more complex software implementation due to addressing and arbitration, bus capacitance limits the number of devices and cable length.
- **Typical Applications:**
    - Connecting to low-speed sensors (e.g., temperature, pressure, humidity).
    - Interfacing with Real-Time Clocks (RTCs).
    - Controlling small character LCDs or OLEDs.
    - Reading/writing to EEPROMs (Electrically Erasable Programmable Read-Only Memory).
    - Communication between microcontrollers on the same board.

Rigs and Simulators

Rigs:

"Rigging" in aircraft refers to the precise **alignment and adjustment of an aircraft's various components, particularly its flight control surfaces and structural elements, to achieve proper flight characteristics, performance, stability, and safety.** It's a critical maintenance process that ensures the aircraft flies straight and level, responds correctly to pilot inputs, and operates efficiently.

Here's a breakdown of what aircraft rigging entails:

Key Aspects of Aircraft Rigging:

- **Early Defect Detection:** SILs enable the discovery of design flaws and software bugs at an initial stage of development.
- **Risk Reduction:** They allow for the safe testing of hazardous or complex operational scenarios.
- **Test Repeatability:** Provides a controlled environment where identical test conditions can be precisely reproduced.
- **Hardware-in-the-Loop (HIL):** Integrates actual physical hardware components with simulated environments or other system elements.
- **Software-Hardware Integration:** Verifies the seamless interaction between software applications and their underlying hardware.
- **Comprehensive Instrumentation:** Features extensive tools for monitoring, recording, and analysing internal system data.
- **Accelerated Development:** Speeds up the iterative design, test, and refinement cycle for complex systems.
- **Lifecycle Support:** Used from initial conceptual design through development, verification, validation, and post-deployment upgrades.
- **Interoperability Verification:** Ensures that all distinct system components and subsystems communicate and function correctly together.
- **Performance Measurement:** Provides a platform for accurately measuring system performance under various conditions.

Why is Rigging Important?

- To integrate and test various subsystems to ensure they work together seamlessly. This means verifying the interfaces, data exchange, and overall functionality of the combined system before it's deployed in the actual aircraft.
- **Performance:** A well-rigged aircraft flies more efficiently, resulting in better fuel economy, higher speeds, and reduced drag.
- **Stability and Control:** Rigging ensures the aircraft is stable in flight and responds predictably and accurately to pilot commands, allowing for smooth and precise manoeuvring.
- **Longevity:** Correct rigging reduces undue stress on aircraft components, extending their lifespan and reducing wear and tear.
- **Compliance:** Aircraft must be rigged according to the manufacturer's specifications and regulatory requirements to maintain their airworthiness.

Rigging is a continuous process throughout an aircraft's life, required after certain maintenance actions, repairs, or if flight characteristics deviate from

normal. It's a testament to the precision engineering and meticulous maintenance required to keep aircraft flying safely and efficiently.

Simulators:

Aircraft simulators are ground-based training devices that precisely replicate the experience of flying a real aircraft. They are indispensable tools in modern aviation for pilot training, aircraft design, research, and even entertainment.

How Aircraft Simulators Work

Aircraft simulators combine various sophisticated technologies to create an immersive and realistic flight environment:

- **Simulation Models:** At their core, simulators use complex mathematical and aerodynamic models to replicate how an aircraft flies, how it reacts to control inputs, the effects of its various systems (engines, hydraulics, avionics), and how it responds to external factors like air density, turbulence, wind, and precipitation.
- **Cockpit Replica:** They feature a highly accurate replica of a specific aircraft's cockpit, including all the controls, instruments, switches, and displays. This ensures pilots develop muscle memory and familiarity with the actual aircraft's layout.
- **Visual System:** High-definition visual displays (often large screens, projector systems, or even virtual reality/mixed reality headsets) recreate the outside world. This includes detailed terrain, airports, other aircraft, weather conditions (day, night, dawn, dusk, fog, clouds), and special effects like lightning or rain. Many professional simulators use collimated displays, which use mirrors to make the projected image appear as if it's at a great distance, enhancing realism.
- **Motion System:** Advanced simulators, particularly "Full Flight Simulators," are mounted on hydraulic or electric motion platforms (often Stewart platforms) that provide cues of acceleration, pitch, roll, and yaw. This simulates the physical sensations of flight, such as take-off and landing forces, turbulence, and turns.

- **Control Loading:** The flight controls (yoke/stick, rudder pedals, thrust levers) provide realistic force feedback to the pilot, mimicking the resistance and feel of the actual aircraft's controls.
- **Sound System:** Realistic sounds from the aircraft's engines, landing gear, alarms, and environmental effects enhance immersion.
- **Instructor Operating Station (IOS):** An instructor can monitor and control the simulation, introduce various scenarios (e.g., system failures, emergencies, different weather conditions, air traffic control interactions), and evaluate the pilot's performance.

Types of Aircraft Simulators

Aircraft simulators are categorized based on their fidelity and purpose, often by regulatory bodies like the FAA (Federal Aviation Administration) or EASA (European Union Aviation Safety Agency):

1. **Full Flight Simulators (FFS):**
    - **Level A to D (Level D being the highest):** These are the most sophisticated and realistic simulators. They are full-size replicas of specific aircraft cockpits, mounted on multi-axis motion platforms, with advanced visual and sound systems.
    - **Purpose:** Used for critical training like type ratings (qualifying to fly a specific aircraft type), initial pilot training, recurrent training, emergency procedure drills, and Multi-Crew Coordination (MCC) training. Level D simulators can even qualify pilots for "zero flight time" (ZFT) training, meaning they can be type-rated on an aircraft without ever flying the real one first.
2. **Flight Training Devices (FTD):**
    - **Levels 1 to 7:** These are less complex than FFS but still highly realistic. They typically replicate aircraft instruments, equipment, panels, and controls but may not have a full motion system or a fully enclosed cockpit.

- **Purpose:** Used for procedural training, instrument flight rules (IFR) training, cockpit familiarization, and practicing specific manoeuvres or emergency checklists. They provide significant training credit towards pilot licenses.

3. **Basic Aviation Training Devices (BATD) and Advanced Aviation Training Devices (AATD):**
   - These are generally lower-cost, fixed-base simulators that offer a realistic representation of an aircraft's controls and displays.
   - **Purpose:** Used for basic flight manoeuvres, instrument training, and building foundational piloting skills. They can provide limited credit towards flight hours for private pilot certification.
4. **Desktop Simulators / PC-based Simulators:**
   - These are software programs that run on home computers, often with specialized joysticks, yokes, and rudder pedals.
   - **Purpose:** Primarily for recreational use, although they can be a valuable tool for aspiring pilots to familiarize themselves with aircraft controls and basic aerodynamics. They are not typically certified for professional training hours.

Benefits of Aircraft Simulators

- **Safety:** Pilots can practice dangerous emergency procedures (e.g., engine failure, system malfunctions, severe weather) without any risk to lives or expensive aircraft.
- **Cost-Effectiveness:** Simulators significantly reduce the need for actual flight hours, saving fuel, maintenance, and operational costs associated with real aircraft.
- **Efficiency and Repetition:** Pilots can practice specific manoeuvres or scenarios repeatedly until mastered, accelerating the learning process.
- **Scenario-Based Training:** Instructors can create highly specific and challenging scenarios (e.g., adverse weather, specific airport approaches, complex system failures) that would be difficult or impossible to replicate in real flight.
- **Standardization:** Simulators provide a consistent training environment, ensuring all pilots receive uniform instruction and meet specific performance standards.
- **Performance Monitoring:** Advanced data logging and recording capabilities allow instructors to monitor pilot performance in detail, provide targeted feedback, and analyse debriefings.
- **Environmental Benefits:** Training in simulators eliminates fuel consumption and reduces noise pollution.
- **Accessibility:** Simulators can be used anytime, regardless of weather conditions or air traffic, allowing for flexible training schedules.

Aircraft simulators continue to evolve with advancements in computing power, graphics, virtual reality, and artificial intelligence, making them an increasingly integral and sophisticated part of aviation training and operations.

PCB Designing:

The Printed Circuit Board (PCB) design process is a systematic workflow that transforms an electronic circuit idea into a physical board, ready for manufacturing. It requires a collaborative effort involving various engineering disciplines like electrical, mechanical, and manufacturing.

Here's a breakdown of the typical PCB design process:

1. Conceptual Design & Requirements Analysis:

- **Define Functionality:** Clearly outline what the electronic product needs to do, its features, and how it interacts with other systems.
- **Component Selection:** Choose the necessary electronic components (microcontrollers, sensors, power supplies, etc.) based on the defined requirements, considering factors like availability, cost, and performance.
- **High-Level Specifications:** Establish preliminary board size, shape, number of layers, power requirements, and environmental considerations (e.g., temperature, humidity, vibration).

2. Schematic Capture:

- **Create Schematic Diagram:** This is the logical blueprint of your circuit. Using Electronic Design Automation (EDA) or Computer-Aided Design (CAD) software, you draw symbols for each component and connect them with "nets" to represent electrical connections.
- **Define Electrical Connections:** Ensure all pin-to-pin connections are accurately represented.
- **Component Library Management:** Create or import accurate symbols and footprints for all components.
- **Initial Simulation (Optional but Recommended):** Simulate parts of the circuit to verify electrical functionality and identify potential issues before physical layout.

3. PCB Layout Design:

- **Board Outline and Mechanical Constraints:** Define the exact physical dimensions, shape, and mounting holes of the PCB based on the enclosure or product requirements.
- **Layer Stack up Definition:** Determine the number of layers (e.g., 2-layer, 4-layer, multi-layer) and the arrangement of signal, ground, and power planes. This is crucial for signal integrity and power distribution.
- **Component Placement:** Strategically place all components on the board. This is a critical step that impacts performance, manufacturability, and thermal management. Key considerations include:
    - Grouping related components (e.g., power section, digital section, analog section).
    - Minimizing trace lengths for high-speed signals.
    - Ensuring proper clearances between components.
    - Considering thermal dissipation for hot components.
    - Adhering to mechanical constraints (e.g., connectors, user interface elements).
- **Routing (Trace Layout):** Connect the components physically by drawing conductive traces on the various layers of the PCB. This is where the nets from the schematic are materialized.
    - **Design Rules:** Apply design rules (trace width, spacing, via sizes, impedance control) to ensure manufacturability and electrical performance.
    - **Critical Signal Routing:** Route high-speed, high-current, or noise-sensitive signals first, often manually, to optimize their path and minimize interference.
    - **Power and Ground Planes:** Implement dedicated power and ground planes for stable power distribution and noise reduction.
    - **Vias:** Use vias to transition traces between different layers.
- **Solder Mask and Silkscreen:**

- **Solder Mask:** Apply a protective layer (usually green) over the copper traces, exposing only the pads where components will be soldered.
- **Silkscreen:** Add text, symbols, and component outlines on the top and bottom of the board for assembly and identification.

4. Design Verification & Validation:

- **Design Rule Check (DRC):** Run automated checks in the EDA software to identify violations of design rules (e.g., short circuits, open circuits, incorrect spacing).
- **Electrical Rule Check (ERC):** Verify the electrical correctness of the schematic and ensure consistency with the layout.
- **Signal Integrity (SI) and Power Integrity (PI) Analysis (for complex designs):** Simulate signal transmission and power distribution to ensure performance and prevent issues like reflections or voltage drops.
- **Design for Manufacturing (DFM) / Design for Assembly (DFA) Review:** Check if the design can be manufactured and assembled efficiently and cost-effectively, considering the capabilities of the chosen fabrication and assembly houses. This often involves collaboration with the manufacturers.

- **Bill of Materials (BOM) Generation:** Create a comprehensive list of all components used in the design, including part numbers, manufacturers, and quantities.

5. Manufacturing Preparation:

- **Gerber File Generation:** Export the PCB design into Gerber files (or ODB++ or IPC-2581), which are industry-standard files that describe each layer of the PCB (copper traces, solder mask, silkscreen, drill holes). These files are sent to the PCB fabrication house.
- **Drill Files:** Generate files that specify the location and size of all holes to be drilled.
- **Pick-and-Place Files:** Generate files that guide automated assembly machines for placing components.
- **Fabrication and Assembly Drawings:** Create detailed drawings for the manufacturer and assembler.

6. PCB Fabrication and Assembly:

- **Fabrication (Bare Board Manufacturing):** The manufacturer uses the design files to etch the copper layers, drill holes, laminate layers together, and apply solder mask and silkscreen.
- **Assembly:** Components are physically placed on the fabricated PCB and soldered, typically using automated pick-and-place machines and reflow ovens.

7. Testing and Debugging:

- **Functional Testing:** Test the assembled PCB to ensure it performs as intended according to the initial specifications.
- **Debugging:** Identify and fix any issues or malfunctions in the circuit.

The PCB design process is iterative, with feedback loops at various stages to ensure a high-quality, functional, and manufacturable product.

System On Chip (SOC):

A **System-on-Chip (SoC)** is an integrated circuit (IC) that combines most or all of the necessary components of a computer or other electronic system onto a single chip. Think of it as an entire system, traditionally built with multiple discrete chips, condensed into one silicon die.

Key components typically found in an SoC include:

- **Central Processing Unit (CPU):** One or more processor cores (e.g., ARM Cortex-A, RISC-V) that execute software instructions.
- **Memory (RAM/ROM):** On-chip memory for storing data and program instructions.
- **Graphics Processing Unit (GPU):** For handling graphical computations.
- **Digital Signal Processors (DSPs):** For specialized signal processing tasks (audio, video).
- **Peripherals:** Various interfaces like USB, Ethernet, Wi-Fi, Bluetooth, SPI, I2C, UART, etc.
- **Memory Controllers:** For external memory (DDR, Flash).
- **Hardware Accelerators:** Dedicated blocks for specific functions to improve performance (e.g., video codecs, cryptography).
- **Interconnects:** Buses (like AXI) that allow all these components to communicate with each other.
- **Power Management Units (PMUs):** For efficient power distribution and management.

SoCs are designed for compact size, low power consumption, high performance, and reduced bill of materials (BOM) cost, making them ubiquitous in mobile phones, tablets, IoT devices, embedded systems, and more.

Smart Fusion Kit:

The **Microchip (formerly Microsemi/Actel) Smart Fusion™ SoC FPGA** is a unique family of System-on-Chip (SoC) Field-Programmable Gate Arrays (FPGAs) that stands out due to its integration of three key elements on a single chip:

- **Kit Used: SmartFusion Soc A2500M3G-1FGG484I**

1. **FPGA Fabric:** A flash-based FPGA fabric for flexible, reconfigurable digital logic.
2. **Microprocessor Subsystem (MSS):** A hard-coded ARM Cortex-M3 processor, providing a robust and efficient general-purpose computing core.
3. **Programmable Analog:** Integrated analog circuitry, including ADCs, DACs, and comparators, enabling true mixed-signal system integration on a single chip.

The "kit" typically refers to **development kits** designed to help engineers evaluate, develop, and prototype designs using Smart Fusion SoC FPGAs.

Key Features of Smart Fusion SoC FPGAs (and why they are unique)

- **Integrated Hard Microprocessor (MSS):** Unlike traditional FPGAs where a processor would have to be implemented in the configurable logic (a "soft-core"), Smart Fusion devices include a dedicated, hard ARM Cortex-M3 processor subsystem. This MSS offers:
    - Higher performance and lower power consumption for software tasks.
    - Standard peripherals like UART, SPI, I2C, Ethernet MAC, USB, CAN, and more.

    - Dedicated memory (eSRAM, eNVM flash memory).
    - A high-bandwidth interconnect (like AXI) to the FPGA fabric.
- **Flash-Based FPGA Fabric:** SmartFusion FPGAs use non-volatile flash technology for their configuration. This provides:
    - **Instant-on capability:** The device boots up immediately, unlike SRAM-based FPGAs that require external configuration memory.
    - **Immunity to Single Event Upsets (SEUs):** Crucial for high-reliability applications (e.g., aerospace, defence, medical) as flash cells are less susceptible to radiation-induced errors.
    - **Enhanced Security:** Flash-based FPGAs offer superior IP protection against cloning, overbuilding, and reverse engineering, with features like secure boot, cryptographic services (AES, SHA, ECC), and Physically Unclonable Functions (PUF).
- **Programmable Analog:** This is a major differentiator. The integrated analog blocks allow designers to create custom mixed-signal solutions on a single chip, reducing board space, complexity, and cost. Features often include:
    - Analog-to-Digital Converters (ADCs) and Digital-to-Analog Converters (DACs).
    - High-speed comparators.
    - Voltage, current, and temperature monitors.
    - An Analog Compute Engine (ACE) to ofload analog processing from the CPU.

Software and Design Tools: Access to Microchip's comprehensive design suite, primarily **Libero® SoC Design Suite**, which provides tools for:

- FPGA design (HDL entry, synthesis, place & route).
- MSS configuration and software development (including ARM Cortex-M3 tools, compilers like Soft Console, Keil, IAR, and RTOS support).
- Analog block configuration.
- Simulation, verification, and debugging.

Software's Used:

Libero SoC Design Suite:

Libero SoC Design Suite is Microchip's comprehensive Integrated Development Environment (IDE) for designing with their flash-based FPGAs, SoC FPGAs (like Smart Fusion and Polar Fire SoC), and radiation-tolerant FPGAs. It serves as the central hub for your hardware design.

Key features and functionalities of Libero SoC Design Suite include:

- **FPGA Design Flow:** It manages the entire FPGA design flow, from RTL (Register Transfer Level) entry (using Verilog or VHDL) through synthesis (using industry-standard Synopsys Synplify Pro®), place & route, timing analysis, and bitstream generation.
- **IP Integration:** It provides a rich IP (Intellectual Property) library for common functions, communication interfaces, and memory controllers. For SoC FPGAs, it also offers specific IPs for configuring the hard microprocessor subsystem (MSS).
- **System Builder:** A graphical tool within Libero that allows you to configure and connect the hard MSS components (like the ARM Cortex-M3 core, memory controllers, and peripherals) with each other and with your custom logic in the FPGA fabric. This is where you define the memory map and interconnects.
- **Programmable Analog Configuration:** For Smart Fusion devices, Libero allows you to configure the integrated analog blocks (ADCs, DACs, comparators).
- **Simulation and Verification:** Integrates with industry-standard simulators like Siemens ModelSim Pro® and QuestaSim Pro® for functional and timing simulation of your hardware design.
- **Debug Capabilities:** Provides on-chip debug tools, including integrated logic analysers, to monitor and debug signals within your FPGA fabric.
- **Constraints Management:** Tools for defining timing, I/O, and physical constraints to ensure your design meets performance requirements and is manufacturable.

- **Secure Production Programming:** Supports secure programming solutions to protect your intellectual property on the flash-based FPGAs.
- **Reference Designs and Development Kits:** Often comes with pre-built reference designs and support for Microchip's development kits, like the SmartFusion2 SoC FPGA Development Kit.

Libero is essential for defining the hardware architecture, including the interaction between the processor and the programmable logic, before you move to software development.

Soft Console:

SoftConsole is Microchip's free, Eclipse-based Integrated Development Environment (IDE) specifically designed for developing C/C++ software for the microprocessor subsystems (MSS) within their SoC FPGAs, such as the ARM Cortex-M3 in Smart Fusion devices or RISC-V cores in Polar Fire SoC.

Here's a breakdown of SoftConsole' s key features and capabilities:

- **Software Development Environment:** It provides a complete environment for writing, compiling, and debugging bare-metal and RTOS-based C/C++ applications. It supports both Windows and Linux operating systems.
- **GNU Toolchain Integration:** SoftConsole leverages the standard GNU GCC (GNU Compiler Collection), GDB (GNU Debugger), and Binutils. This provides a familiar and powerful set of tools for embedded software development.
- **Debugging Capabilities:** It offers robust debugging features including:
    - Setting breakpoints and single-stepping through code.
    - Inspecting memory contents and CPU registers.
    - Real-time debugging on the target hardware (SoC FPGA) via JTAG using Microchip's FlashPro programmers (FlashPro3, FlashPro4, FlashPro5, FlashPro6).
    - Integration with OpenOCD for JTAG debugging.
- **RTOS Support:** SoftConsole supports popular Real-Time Operating Systems (RTOS) to manage complex, multi-tasking embedded software applications.

- **Processor Support:** It specifically supports Microchip's SoC FPGAs with hard-coded processor cores like the ARM Cortex-M3 (found in SmartFusion) and various RISC-V CPU soft cores and hard cores (found in PolarFire SoC, RTG4, IGLOO2, SmartFusion2).
- **Eclipse-Based:** Being built on Eclipse, it offers a customizable and extensible interface that many developers are already familiar with.
- **Hardware Abstraction Layer (HAL) and Drivers:** When you export hardware information from Libero SoC (the hardware design tool), SoftConsole automatically generates or includes necessary Hardware Abstraction Layer (HAL) files and peripheral drivers. These facilitate easy interaction between your C/C++ code and the MSS peripherals, as well as any custom IP instantiated in the FPGA fabric.
- **Project Management:** SoftConsole provides tools to manage your software projects, organize source files, configure build settings (pre-processor, compiler, linker), and manage libraries.
- **Emulator Support:** Recent versions of SoftConsole include support for RISC-V emulation using Antmicro Renode™, allowing for software development and testing even before hardware is available.
- **Security Features:** It can be used to develop and debug code that leverages the security features of Microchip's flash-based FPGAs, such as secure boot and cryptographic functions.

In essence, SoftConsole is the software development counterpart to Libero SoC Design Suite, forming a complete ecosystem for designing and implementing embedded systems on Microchip's SoC FPGAs.

LED blinking using Verilog and Results

Clock Generation:

The clock is a fundamental component in nearly all digital electronic systems, from microcontrollers and microprocessors to FPGAs and complex System-on-Chips (SoCs). Its primary purpose is to provide synchronization and timing for all operations within the system.

Need of Clock:

The clock is essential in digital circuits for several critical reasons:

- **Synchronization:** Digital systems operate by performing actions in a specific order and at precise moments. A clock signal acts as a metronome, synchronizing the operations of various components (like flip-flops, registers, and logic gates) so they all "act" or "change state" at the same predefined instances. Without a common clock, different parts of the circuit would operate asynchronously, leading to unpredictable behaviour, data corruption, and system failures.
- **Sequential Logic Operation:** Many digital circuits are sequential, meaning their output depends not only on the current inputs but also on the sequence of past inputs (i.e., their internal state). Flip-flops and registers, the building blocks of memory and state machines, are sensitive to clock edges. The clock signal dictates when these elements capture new data and transition to a new state.
- **Data Transfer and Reliability:** Data moves between different parts of a digital system. The clock ensures that data is stable and valid at the source before it's sampled by the destination, preventing setup and hold time violations which can lead to metastable states or incorrect data capture.
- **Performance Measurement:** The clock frequency (how many cycles per second) directly relates to the speed at which a digital system can

operate. A higher clock frequency generally allows for more operations per second, thus increasing the system's performance.

Usage of the Clock for the LED Verilog Code:

The clock used by the Kit might not be suitable for the led to blink because the frequency, Blink rate is increased significantly and is not seen by the human eye and so the clock has to be slow and therefore an internal clock is required so that the LED blinking can be seen. In essence, divides down the fast clk to produce a slow periodic signal (led) that acts as a dedicated clock for the LED blinking behaviour.

Verilog Code for 1 sec Blinking of LED:

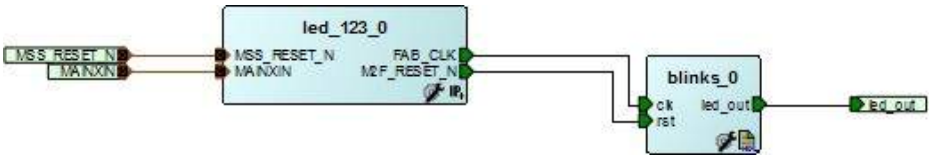
Main Code:

```
module led_s(input wire clk,
input wire rst_n,
output reg led_out
);
parameter clk_freq=5000000;
parameter blink_rate_hz=1;
reg[31:0] counter;
localparam period_cycles=clk_freq/blink_rate_hz;
localparam half_period_cycles=period_cycles/2;
always@(posedge clk or negedge rst_n)
begin
if(~rst_n)
begin
counter<=40'b0;
led_out<=1'b0;
end
else
begin
if(counter==half_period_cycles-1)
begin
counter<=40'b0;
led_out<=~led_out;
end
else
begin
counter<=counter+1'b1;
end
end
end
endmodule
```

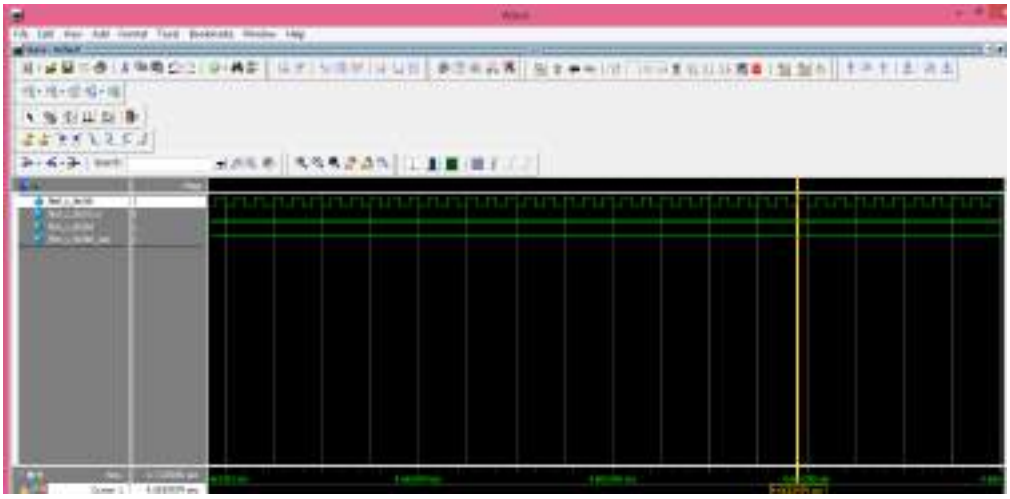
Test Bench:

```
1   `timescale 1ns/1ps
2
3   module led_s_tb;
4   reg clk, rst_n;
5   wire led;
6
7   led_s uut(.clk(clk), .rst_n(rst_n), .led_out(led_out));
8
9   initial begin
10      clk = 0;
11      rst_n = 1;
12
13      #1000 rst_n = 0;
14      #100000000;
15      $finish;
16  end
17
18  initial forever #100 clk = ~clk;
19
20  endmodule
```

Smart Design:



Simulation Results:



Verilog Code for 1,2 and 4 sec Blinking of LED:

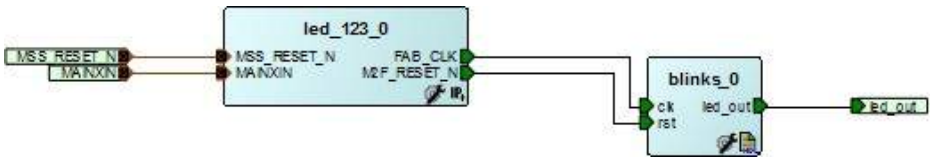
Main Code:

```
module blinks(input wire clk,
input wire rst,
output reg led_out
);
parameter clk_freq=500000;
reg clk_1Hz = 1;
reg [31:0] counter_main = 0;

always @(posedge clk) begin
    if(clkout) begin
        clk_1Hz = 1;
        counter_main = 0;
    end
    else if(counter_main == clk_freq/2 - 1) begin
        clk_1Hz = ~clk_1Hz;
        counter_main = 0;
    end
    else counter_main = counter_main + 1'b1;
end
reg clk_1 = 1, clk_2 = 1, clk_4 = 1;

always @(posedge clk_1Hz) begin
    clk_1 = ~clk_1;
end

always @(posedge clk_1) begin
    clk_2 = ~clk_2;
end

always @(posedge clk_2) begin
    clk_4 = ~clk_4;
end

reg [3:0] counter_60 = 0;
reg [1:0] state_124 = 2'b00;

always @(posedge clk_1Hz) begin
    if(0) begin
        counter_60 = 0;
        state_124 = 2'b00;
    end
    else if(counter_60 == 2'b10) begin
        counter_60 = 0;
        if(state_124 == 2'b10) state_124 = 2'b00;
        else state_124 = state_124 + 2'b01;
    end
    else counter_60 = counter_60 + 2'b01;
end

always @(posedge clk_1Hz) begin
    if(0) led_out = 1;
    else begin
        case(state_124)
            2'b00: led_out = clk_1;
            2'b01: led_out = clk_2;
            2'b10: led_out = clk_4;
            default: led_out = 0;
        endcase
    end
end

endmodule
```

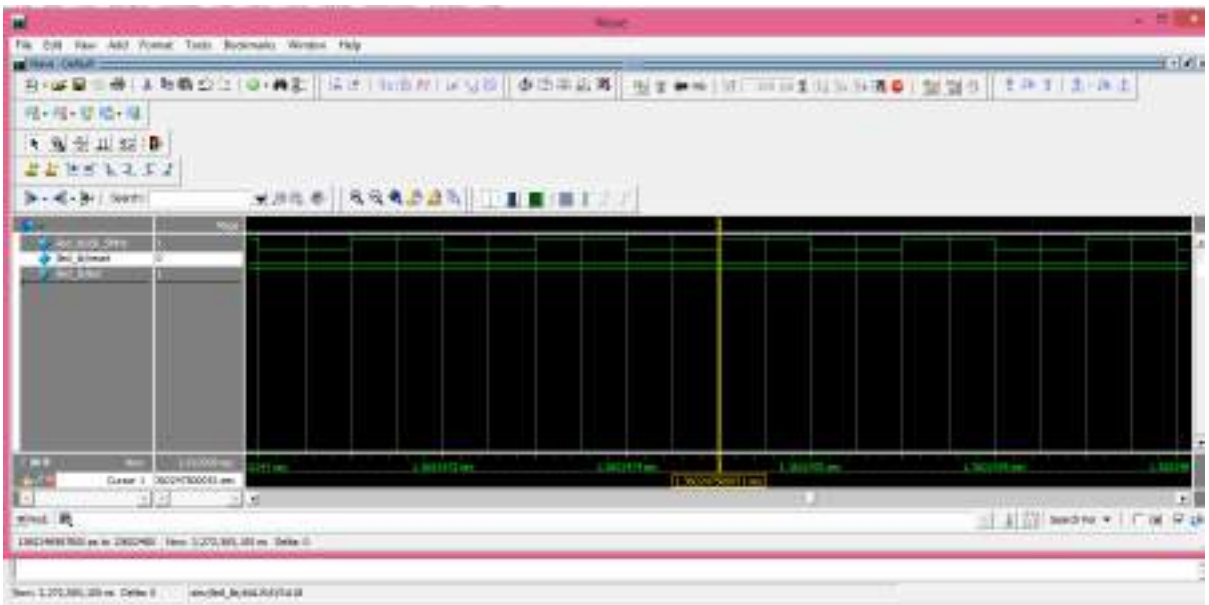
Teat Bench:

```
`timescale 1ns/1ps
module led_tb;
reg clk_5MHz, reset;
wire led;
led_124_ x(.clk(clk_5MHz), .reset(reset), .led_out(led));
initial begin
    clk_5MHz = 0;
    reset = 0;
#1000 reset = 1;
    #100000000;
    //$finish;
end
always #100 clk_5MHz = ~clk_5MHz;
endmodule
```

Smart Design:



Simulation Results:



Coding MSS using C

Tera Term Software:

Tera Term is a free, open-source terminal emulator program for Microsoft Windows. It's widely used in electronics, embedded systems development, and network administration for serial communication and remote logins.

1. Why is it used?

Tera Term is used for several key purposes:

- **Serial Communication (UART/RS-232):** This is one of its most common uses in embedded systems and electronics. It allows a PC to communicate directly with microcontrollers, development boards, and other devices via a serial port (like a USB-to-UART converter). This enables:
    - **Debugging:** Viewing debugging messages, sensor readings, or program output sent from a device.
    - **Controlling Devices:** Sending commands or input to a device.
    - **Firmware Upload:** Some bootloaders or firmware update tools use serial communication.
- **Terminal Emulation (SSH, Telnet):** Tera Term also functions as a client for network protocols like SSH (Secure Shell) and Telnet. This allows users to:
    - Remotely log in to servers, routers, and other network devices.
    - Execute commands on remote systems.
    - Transfer files securely (using SSH/SFTP).
- **Simplicity and Lightweight:** It's a relatively small program with a straightforward interface, making it quick to download and easy to use for basic communication tasks.
- **Customization:** It offers various customization options for terminal settings, fonts, colors, and macros, which can be useful for repetitive tasks.
- **Logging:** It can log communication sessions to a file, which is helpful for debugging or record-keeping.

2. Which COM port to be used?

When using Tera Term for serial communication, you need to select the correct **COM port** that corresponds to your device's serial connection.

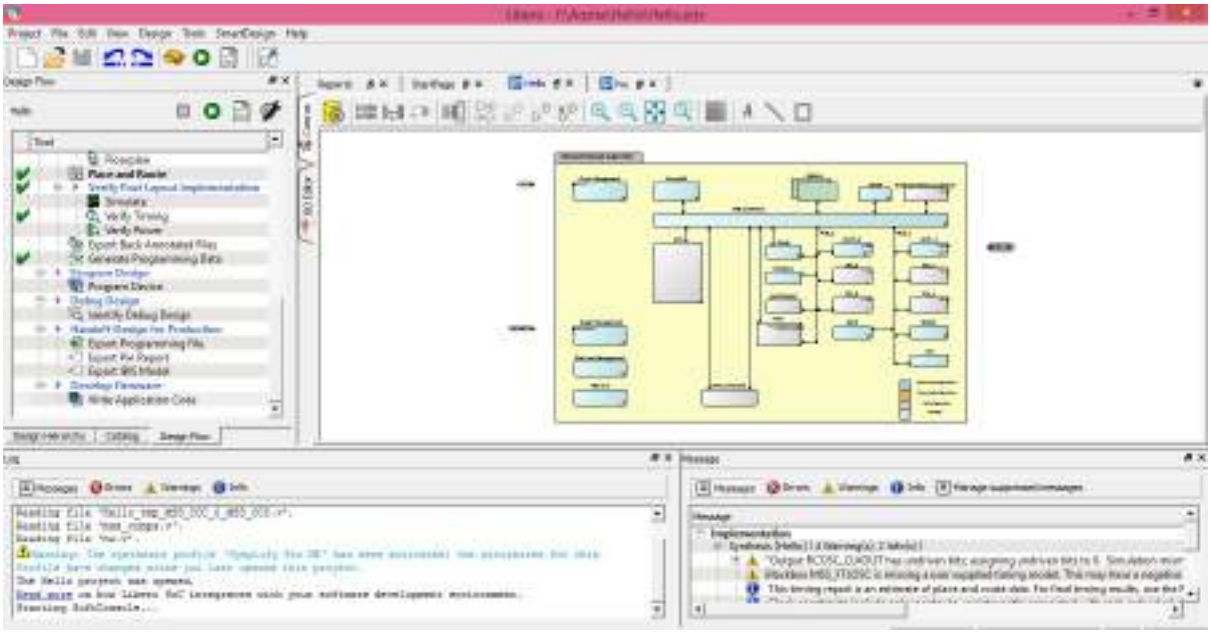
- **Identifying the COM Port:**
    1. **Connect your device:** Plug in your microcontroller, development board, or USB-to-UART converter to your computer's USB port.
    2. **Device Manager (Windows):** Open the Windows Device Manager. You can usually find it by searching "Device Manager" in the Start Menu.
    3. **Ports (COM & LPT):** In Device Manager, expand the "Ports (COM & LPT)" section.
    4. **Look for your device:** You should see an entry for your device, often labelled something like "USB Serial Port (COMX)" or "USB-SERIAL CH340 (COMX)," "FT232R USB UART (COMX)," or similar, where "X" is the COM port number (e.g., COM3, COM4, COM10).
    5. **Note the number:** This is the COM port number you will select in Tera Term.
- **Selecting in Tera Term:**
    1. Open Tera Term.
    2. In the "New connection" dialog box (or by going to **File > New connection...**), select **"Serial"**.
    3. From the dropdown list, choose the **COM port number** you identified in the Device Manager.
    4. Click **"OK"**.
    5. Go to **Setup > Serial port...** to configure the serial port settings (Baud rate, Data bits, Parity, stop bits, Flow control) to match your device's requirements. This information is usually provided in your device's documentation or firmware code.

Printing Hello World using Tera Term:

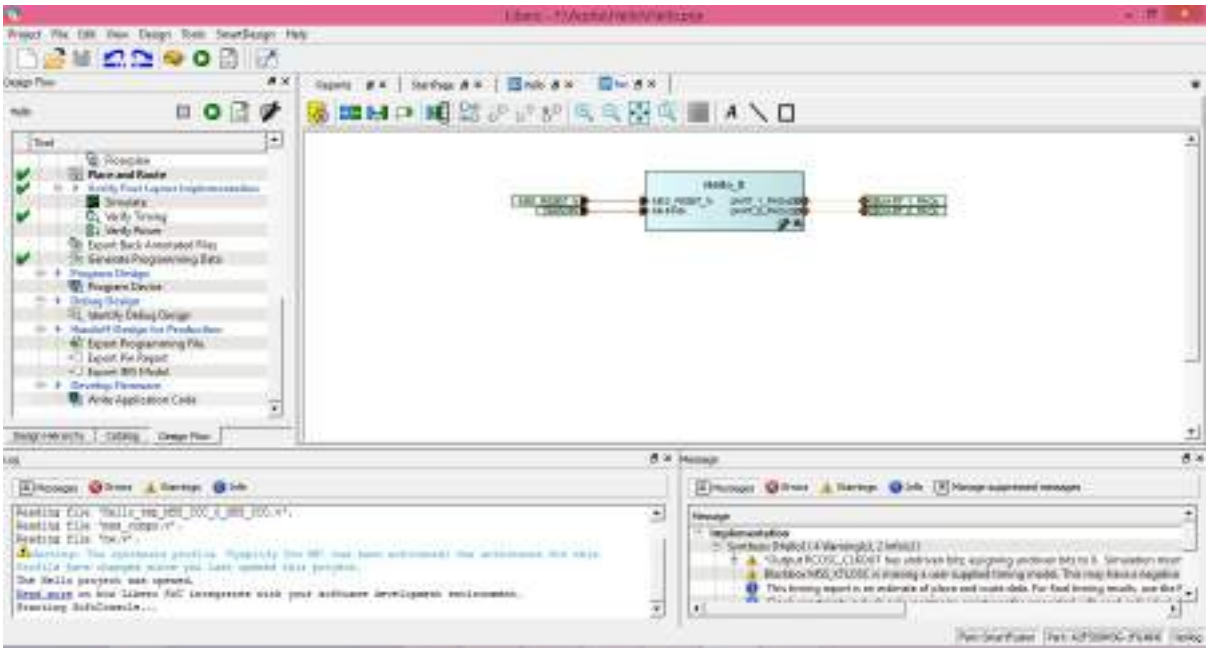
Soft Console Code:

```
#include "mss_uart.h"

int main(void)
{
    uint8_t message[12] = "Hello World";

    MSS_UART_init( &g_mss_uart0, MSS_UART_57600_BAUD,
                   MSS_UART_DATA_8_BITS | MSS_UART_NO_PARITY | MSS_UART_ONE_STOP_BIT );
    MSS_UART_polled_tx_string( &g_mss_uart0, message );
    return(0);
}

void
MSS_UART_polled_tx_string
(
    mss_uart_instance_t * this_uart,
    const uint8_t * p_sz_string
);
```

MSS Configuration:



Smart Design:



Tera Term Output:

Choice based blinking of LED with Pattern (1,2 and 4 Sec Interval)

Combining both FPGA block and MSS block using APB bus in the Smart Fusion Kit:

- **Simplicity:** APB is a straightforward bus protocol, making it relatively easy to implement APB slave interfaces for custom peripherals in the FPGA fabric. This is less complex than implementing a full AHB or AXI slave.
- **Control/Configuration:** It's well-suited for control and status register access, configuring peripherals, and initiating data transfers (even if the actual high-speed data path uses a different mechanism, the control for it can be via APB).
- **Memory-Mapped Access:** Provides a standard memory-mapped interface, making it easy for the Cortex-M3 software to interact with the custom hardware using simple pointer dereferences.
- **Resource Efficiency:** Its simpler design consumes fewer FPGA resources compared to higher-bandwidth buses.

APB Bridge/Interconnect:

- Within the SmartFusion design environment (e.g., Microchip's Libero SoC Design Suite), you'll instantiate an APB bridge or interconnect component. This bridge connects to the MSS's internal bus (AHB-Lite usually acts as the master) on one side and provides multiple APB slave interfaces on the other.
- The APB bridge handles the address decoding and routing of transactions from the MSS to the correct custom peripheral in the FPGA fabric.

The APB bus in a SmartFusion Kit provides a standardized, straightforward, and efficient way for the embedded ARM Cortex-M3 microcontroller to communicate with and control custom hardware peripherals that you design and implement within the FPGA fabric.

Verilog Code:

Main Code:

```
module blink(clk,rst,led_out,
apb_rst,
apb_psel,
apb_paddr,
apb_penable,
apb_pwrite,
apb_pwdata,
apb_prdata,
apb_pready,
apb_pslverr
);
input wire clk;
input wire rst;
output reg led_out;
input apb_rst;
input apb_psel;
input[31:0] apb_paddr;
input apb_penable;
input apb_pwrite;
input[31:0] apb_pwdata;
output reg[31:0] apb_prdata;
output reg apb_pready;
output reg apb_pslverr;
parameter clk_freq=5000000;
reg clk_1Hz = 0;
reg [31:0] counter_main = 0;
reg[3:0] choice = 0;
always @(posedge clk) begin
    if(clk==0) begin
        clk_1Hz = 0;
        counter_main = 0;
    end
    else if(counter_main == clk_freq/2 - 1) begin
        clk_1Hz = ~clk_1Hz;
        counter_main = 0;
    end
    else counter_main = counter_main + 1'b1;
end
reg clk_1 = 0, clk_2 = 0, clk_4 = 0;

always @(posedge clk_1Hz) begin
    clk_1 = ~clk_1;
end

always @(posedge clk_1) begin
    clk_2 = ~clk_2;
```

```
47  end
48
49  always @(posedge clk_2) begin
50      clk_4 = ~clk_4;
51  end
52
53  always @(posedge clk_1Hz) begin
54      if(apb_rst==1&&apb_psel==1&&apb_penable==1&&apb_pwrite==1)
55      begin
56      if(apb_paddr==32'h40050000)
57      begin
58      choice<=apb_pwdata;
59      end
60
61      else begin
62        choice<=0;
63  end
64
65  case(choice)
66              2'b00: led_out = clk_1;
67              2'b01: led_out = clk_2;
68              2'b10: led_out = clk_4;
69              default: led_out =1;
70          endcase
71      end
72  end
73  endmodule
74
```

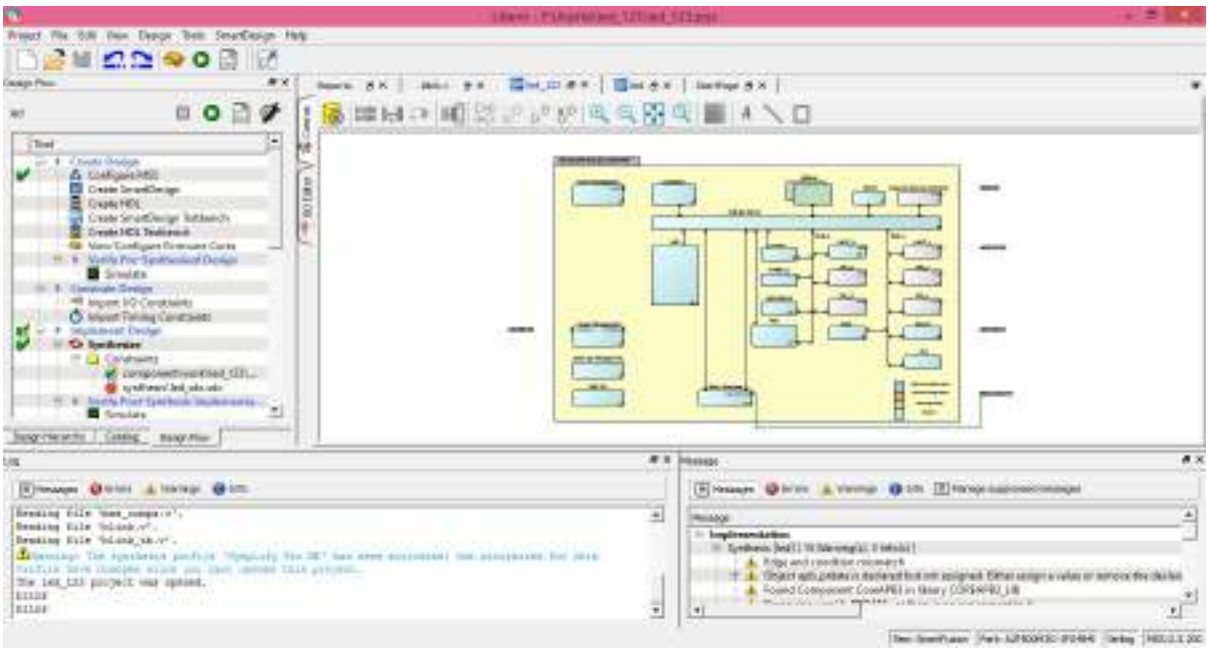
Test Bench:

```
module tb_top();
    reg clk;
    reg rst;
    wire led_out;
    reg apb_rst;
    reg apb_psel;
    reg[31:0] apb_paddr;
    reg apb_penable;
    reg apb_pwrite;
    reg[31:0] apb_pwdata;
    wire[31:0] apb_prdata;
    wire apb_pready;
    wire apb_pslverr;

top_apb u0(.clk(clk),.rst(rst),.led_out(led_out),.apb_rst(apb_rst),.apb_psel(apb_psel),.apb_paddr(apb_paddr),.apb_penable(apb_penable),.apb_pwrite(apb_pwrite),
.apb_pwdata(apb_pwdata),.apb_prdata(apb_prdata),.apb_pready(apb_pready),.apb_pslverr(apb_pslverr));
parameter CLOCK_FREQ=[illegible];
parameter clk_period=[illegible];
initial begin
clk=1;
forever #(clk_period/2) clk=~clk;
end
initial begin
rst=1'b1;
apb_rst=1'b1;
apb_psel=1'b0;
apb_penable=1'b0;
apb_pwrite=1'b1;
apb_paddr=32'h40050000;
apb_pwdata=32'h0000;
#[illegible]
rst=1'b0;
apb_rst=1'b0;
#[illegible]
apb_psel=1'b1;
apb_penable=1'b0;
#[illegible]
apb_paddr=[illegible];
apb_pwdata=32'h0000;
apb_penable=1'b1;
#[illegible]
apb_pwrite=1'b0;
#[illegible]
apb_paddr=32'h4005[illegible];
apb_pwdata=[illegible];
```

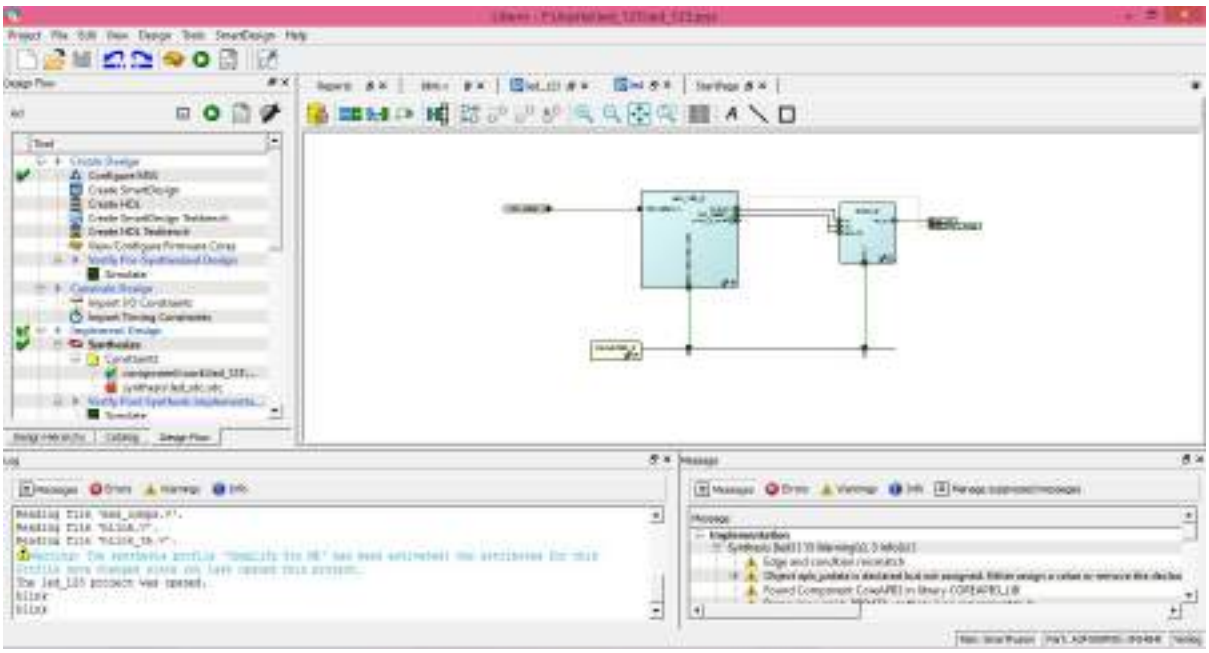
Soft Console:

- The baud rate defines the **rate at which serial data is transmitted and received** over a communication channel. It is measured in bits per second (bps) and specifies how many signal changes (or symbols) occur per second. For simple UART communication, one symbol often corresponds to one bit.
- Proper baud rate configuration ensures that both the transmitting and receiving devices are synchronized in their timing, allowing them to correctly interpret the stream of bits.
- This helps to show prints in the Tera Term.

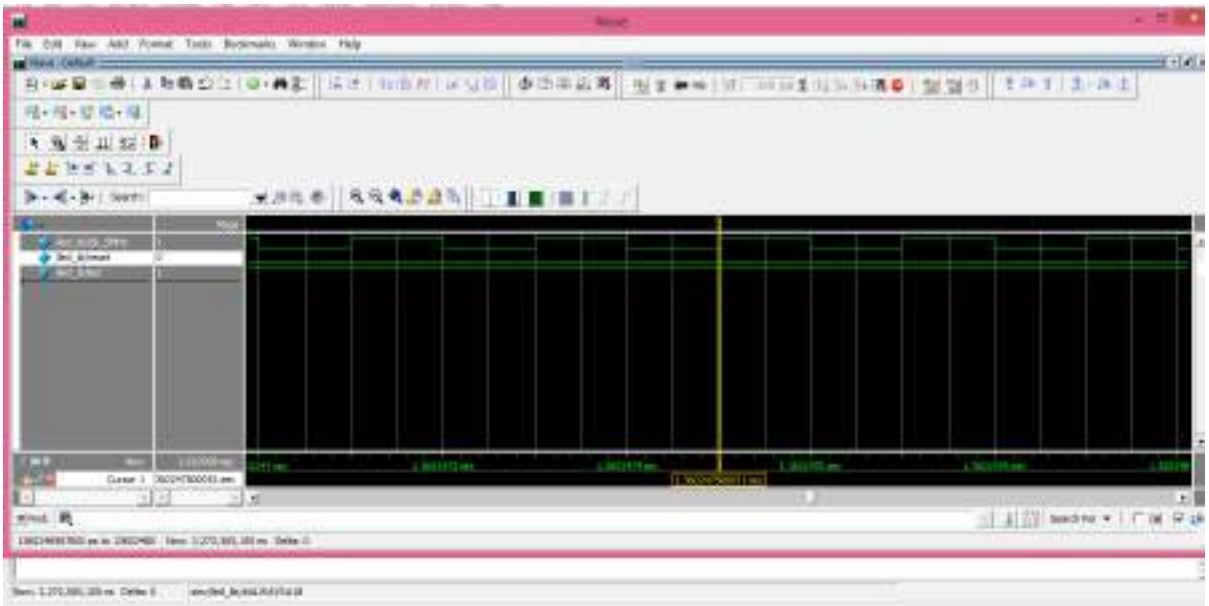
MSS configuration:



Smart Design:



Simulation Results:



Tera Term:

Challenges and Solutions

Technical Challenges:

- The initial implementation resulted in the LED on the System-on-Chip (SoC) blinking at an unmanageably high frequency.
- Configuring the Microprocessor Subsystem (MSS) on the System-on-Chip (SoC) presented a significant hurdle.
- A mismatch between the Baud rate and the Microprocessor Subsystem (MSS) frequency prevented successful communication, resulting in no visible output in Tera Term.
- The initial schematics for the voltage regulator did not meet the specified design requirements.

Solutions:

- To address this, an internal clock was created within the SoC design, enabling precise control over the LED's blinking intervals.
- This challenge was overcome by thoroughly reviewing the comprehensive documentation provided, which then allowed for the successful execution of the necessary configuration operations.
- This issue was resolved by configuring the Baud rate correctly, following the instructions detailed in the documentation and with assistance from the provided guide.
- To ensure correct interpretation and implementation, the data sheet was thoroughly utilized, along with valuable assistance from the guide.

Impact and Contributions

Personal Impact:

My internship at Hindustan Aeronautics Limited (HAL) significantly broadened my engineering expertise, particularly in the realm of avionics. I gained hands-on experience with critical tools and technologies such as **Libero SoC Design Suite**, **SoftConsole**, and practical applications of **voltage regulators** and **SmartFusion System-on-Chip (SoC)** devices. This immersive experience was instrumental in developing and enhancing my skills in **PCB designing** and understanding the intricate workings of **Mission Computers** and other essential aircraft components. Furthermore, I acquired invaluable knowledge about complex aerospace engineering practices, including the use of **rigs and simulators** for system validation, and various other fundamental **aircraft-related concepts**. This exposure reshaped my approach to engineering challenges, empowering me to explore and implement diverse methodologies in problem-solving.

Contributions:

I have not made any contributions but during my internship, my primary projects involved developing a choice-based LED blinking application utilizing the **SmartFusion kit**, which provided practical experience in embedded system design. Additionally, I drew **voltage regulator schematics** tailored for specific voltage differences, ensuring precise interpretation of design requirements.

Conclusion and Future Work

Conclusion:

This internship at Hindustan Aeronautics Limited (HAL) provided an invaluable hands-on experience in the intricate world of avionics and embedded systems. I successfully navigated practical challenges such as optimizing LED blinking frequencies by implementing internal clocks, effectively configuring the Microprocessor Subsystem (MSS) on the SmartFusion SoC through diligent documentation review, and resolving communication issues by correctly setting Baud rates. Furthermore, the task of accurately interpreting and drawing voltage regulator schematics for specific requirements significantly enhanced my practical understanding of power electronics design.

The projects deepened my proficiency with essential tools like Libero SoC Design Suite and SoftConsole, while providing concrete experience in PCB designing. Crucially, I gained a foundational understanding of Mission Computers and the diverse components integral to modern aircraft. This exposure also shed light on critical aerospace engineering practices, including the importance of test rigs, simulators, and broader aircraft concepts. Overall, the internship cultivated an adaptive mindset, empowering me to approach complex engineering challenges with diverse and effective methodologies.

Future Work:

Building upon the foundational knowledge and practical skills acquired, future work could explore several exciting avenues:

1. **Advanced SmartFusion SoC Development:** Further leverage the SmartFusion kit to implement more sophisticated functionalities, such as integrating various sensors, establishing complex communication protocols (e.g., CAN, Ethernet), or developing real-time control applications within the FPGA fabric. This would involve a deeper dive into custom IP creation and hardware-software co-design.
2. **Voltage Regulator Prototyping and Testing:** Progress from schematic design to the actual fabrication of the voltage regulator PCBs. This would involve hands-on soldering, thorough testing of their performance under varying load conditions, and detailed thermal analysis to ensure robustness and efficiency.
3. **Expanded Avionics System Understanding:** Engage in projects that explore higher-level avionics system integration, potentially within an Avionics Integration Lab (AIL) or by studying "Iron Bird" methodologies. This would provide insight into the holistic interaction between mission computers and other flight-critical systems.
4. **Deep Dive into RTOS and Security:** Investigate the deployment of Real-Time Operating Systems (RTOS) on the SmartFusion SoC, exploring task scheduling, memory management, and inter-process communication. Additionally, understanding the robust security features inherent in flash-based FPGAs could lead to projects focused on secure boot and data protection for mission-critical applications.

References:

- Microchip Technology Inc., "SmartFusion SoC FPGA Data Sheet," Document No. D-1592206.
- Microchip Technology Inc., "Libero® SoC Design Suite," [11.9].
- Microchip Technology Inc., "SoftConsole User Manual," [3.4v].
- Texas Instruments, "TPS7A4701RGWR Datasheet" (for LDO Regulator).
- Texas Instruments, "TPS5431DDA Datasheet" (for Buck Converter).
- Tera Term.

CERTIFICATE

ಆಯುಧಗಳು ಮತ್ತು ಸಮರ ವ್ಯವಸ್ಥೆ ಸಂಶೋಧನೆ ಮತ್ತು ವಿನ್ಯಾಸ ಕೇಂದ್ರ
मिशन एवं युद्धक प्रणाली अनुसंधान एवं अभिकल्प केंद्र
MISSION & COMBAT SYSTEM RESEARCH & DESIGN CENTRE
ವಿನ್ಯಾಸ ಸಂಕೀರ್ಣ / अभिकल्प कॉम्प्लेक्स / DESIGN COMPLEX
ಹಿಂದೂಸ್ಥಾನ್ ಏರೋನಾಟಿಕ್ಸ್ ಲಿಮಿಟೆಡ್
हिन्दुस्तान एरोनॉटिक्स लिमिटेड
HINDUSTAN AERONAUTICS LIMITED

ಮಾರತ್ತ ಹಳ್ಳಿ ಪೋಸ್ಟ್, ಬೆಂಗಳೂರು-560 037, ಭಾರತ
मारतहल्ली पोस्ट, बेंगलूरु - 560 037, भारत
Marathahalli Post, Bengaluru - 560 037, India

No. D/MCSRDC/HR/Internship/708/2025 Date: 27.06.2025

CERTIFICATE

This is to certify that the following student of **B Tech(ECE), IV SEMESTER,** from **M/s VELLORE INSTITUTE OF TECHNOLOGY, VELLORE** has completed his **"INTERNSHIP"** at MCSRDC, Design Complex, HAL, Bengaluru from **12.05.2025 to 27.06.2025** on "no-pay-no-fee-basis".

Mr. ROHAN YENIGALLA (23BEC0027)

The punctuality, conduct, behavior and overall progress of the student were **EXCELLENT** during the training.

27/06/25

(M Kathirvel)
Dy. General Manager (HR)
ಉಪ ಮಹಾಪ್ರಬಂಧಕ / उप महाप्रबंधक / M KATHIRVEL
ಮಹಾ ಪ್ರಬಂಧಕರು / महाप्रबंधक / DGM (HR)
ಎಂಸಿಎಸ್‌ಆರ್‌ಡಿಸಿ (ಎಚ್‌ಆರ್) / एमसीएसआरडीसी (एचआर) / MCSRDC (DC)
ಎಚ್‌ಎಎಲ್, ಬೆಂಗಳೂರು / एचएएल, बेंगलूरु / HAL, BANGALORE-560 037

HAL MCSRDC HR DEPARTMENT

ನೋಂದಾಯಿತ ಕಛೇರಿ : 15/1, ಕಬ್ಬನ್ ರಸ್ತೆ, ಬೆಂಗಳೂರು - 560 001, ಭಾರತ
पंजीकृत कार्यालय : 15 /1, कब्बन रोड, बेंगलूरु - 560 001, भारत
Registered Office : 15 /1, Cubbon Road, Bengaluru - 560 001, India
CIN: L35301KA1963GOI001622

आज़ादी का अमृत महोत्सव